

Comprehensive C# Inheritance Notes (Full Guide)

This full guide focuses exclusively on **inheritance in C#** (.NET 8+ compatible), expanding on fundamentals to advanced concepts. It's structured per your outline for easy navigation. Each section includes detailed explanations, syntax, rules, diagrams (text-based), and executable code examples with outputs. Inheritance promotes code reuse and polymorphism via “is-a” relationships. All code is console-app ready—copy-paste into a `Program.cs` for testing.

Key C# Principles: Single inheritance for classes; multiple via interfaces. Supports runtime polymorphism. Use `virtual/override` for flexibility.

1. Basics of Inheritance

Definition and Purpose of Inheritance

Inheritance allows a class (derived/child) to inherit members (fields, properties, methods) from another class (base/parent). It establishes an “is-a” hierarchy (e.g., *Dog is-a Animal*).

Purpose: - **Reusability:** Share common code (e.g., all animals eat). - **Extensibility:** Add/specialize without altering base. - **Polymorphism:** Use base references for derived objects (runtime method dispatch). - **Abstraction:** Define contracts for subclasses.

Need and Advantages of Inheritance

Need: Without it, duplicate code across similar classes (e.g., `Car` and `Truck` both need `Drive()`). Leads to bugs and maintenance issues in scalable apps.

Advantages: - **DRY Principle:** Don't Repeat Yourself—centralize shared logic. - **Maintainability:** Update base once, affects all derived. - **Testability:** Test base behaviors uniformly. - **Design:** Models real-world hierarchies (e.g., UI controls).

Potential Drawbacks: Tight coupling; deep hierarchies complicate debugging. Mitigate with interfaces.

Real-World Examples of Inheritance

- **Employee System:** `Employee` (base: `Salary`, `Work()`) → `Manager` (derived: `Team`, `Delegate()`).
- **Shapes in Graphics:** `Shape` (base: `Draw()`, `Area()`) → `Circle/Rectangle` (derived: specific calculations).
- **Vehicles in Logistics:** `Vehicle` (base: `Speed`, `Move()`) → `Car/Truck` (derived: `LoadCapacity`).

Syntax of Inheritance (: in C#)

Use : to inherit. Base class first; derived extends it.

Basic Syntax:

```
public class BaseClass // Base
{
    // Members
}

public class DerivedClass : BaseClass // Derived
{
    // Inherited + new members
}
```

Example (Full Code with Output):

```
using System;

public class Animal // Base
{
    public string Name { get; set; }
    public Animal(string name) { Name = name; }
    public virtual void Speak() { Console.WriteLine($"{Name} makes a sound."); }
}

public class Dog : Animal // Derived inherits Speak(), Name
{
    public Dog(string name) : base(name) { }
    public override void Speak() // Override for polymorphism
    {
        Console.WriteLine($"{Name} barks: Woof!");
    }
}

class Program
{
    static void Main()
    {
        Animal generic = new Dog("Rex"); // Upcast
        generic.Speak(); // Output: Rex barks: Woof! (polymorphic)
    }
}
```

Output:

Rex barks: Woof!

2. Types of Inheritance

C# supports single, multilevel, and hierarchical inheritance directly for classes. Multiple and hybrid use interfaces (no diamond ambiguity). Enums/structs don't inherit classes.

Type	Description	Supported in C#?
Single Inheritance	One class inherits from one base	(direct)
Multilevel Inheritance	Chain: $A \rightarrow B \rightarrow C$	(direct)
Hierarchical Inheritance	One base $\rightarrow$ multiple derived (e.g., tree)	(direct)
Multiple Inheritance	One class from multiple bases	(classes; via interfaces)
Hybrid Inheritance	Mix (e.g., hierarchical + multiple)	(via interfaces)

Single Example:

```
public class Vehicle { public void Start() { } }  
public class Car : Vehicle { } // Single
```

Multilevel Example:

```
public class Animal { }  
public class Mammal : Animal { }  
public class Dog : Mammal { } // Dog inherits Animal indirectly
```

Hierarchical Example:

```
public class Shape { }  
public class Circle : Shape { }  
public class Square : Shape { } // Both inherit Shape
```

Multiple via Interfaces:

```
interface IFlyable { void Fly(); }  
interface ISwimmable { void Swim(); }  
public class Duck : Animal, IFlyable, ISwimmable { } // Multiple capabilities
```

3. Access Modifiers and Inheritance

C# modifiers: **public** (everywhere), **private** (class-only), **protected** (class + derived), **internal** (assembly), **protected internal** (assembly or derived),

`private protected` (class/derived in assembly).

How Accessibility Affects Inherited Members

- Inherited: `public`, `protected`, `internal` (if same assembly).
- Not Inherited: `private`.
- Derived can override/ hide inherited members but can't reduce visibility (e.g., `public` base can't become `private` derived).

Protected Members — Most Used in Inheritance

`protected`: Base exposes internals to derived without public leak. Ideal for customization (e.g., `protected` fields for subclass logic).

Example:

```
using System;
```

```
public class BankAccount // Base
{
    protected decimal balance; // Protected: Derived access
    public BankAccount(decimal initial) { balance = initial; }
    public virtual void Deposit(decimal amount)
    {
        balance += amount;
        Console.WriteLine($"Deposited: {amount}, Balance: {balance}");
    }
}
```

```
public class SavingsAccount : BankAccount // Derived
{
    private decimal interestRate = 0.05m;
    public SavingsAccount(decimal initial) : base(initial) { }
    public override void Deposit(decimal amount)
    {
        base.Deposit(amount); // Use base
        balance += balance * interestRate; // Protected access + custom
        Console.WriteLine($"Interest added. New Balance: {balance}");
    }
}
```

```
class Program
{
    static void Main()
    {
        SavingsAccount acct = new SavingsAccount(100);
        acct.Deposit(50); // Output: Deposited: 50, Balance: 150 \n Interest added. New Ba
```

```
    }
}
```

Output:

```
Deposited: 50, Balance: 150
Interest added. New Balance: 157.5
```

4. Base and Derived Classes

Creating Base and Derived Classes

Base: Common members. Derived: `class Derived : Base { } + specifics.`

Constructor Execution Order (Base → Derived)

1. Base static fields/ctors.
2. Base instance ctor.
3. Derived instance ctor. (Fields initialized before ctors.)

Using the base Keyword (C#)

`base()` for ctors; `base.Method()` for members.

Invoking Base Class Methods or Constructors

Explicit for chaining/overriding.

Example (Constructor Chaining + Method Invoke):

```
using System;

public class Vehicle // Base
{
    public string Brand;
    public Vehicle(string brand)
    {
        Brand = brand;
        Console.WriteLine($"Vehicle created: {Brand}");
    }
    public virtual void Accelerate() { Console.WriteLine("Vehicle accelerating."); }
}

public class ElectricCar : Vehicle // Derived
{
    public int BatteryLevel;
    public ElectricCar(string brand, int battery) : base(brand) // base ctor
    {
```

```

        BatteryLevel = battery;
        Console.WriteLine($"ElectricCar ready with {BatteryLevel}% battery.");
    }
    public override void Accelerate()
    {
        base.Accelerate(); // Invoke base
        Console.WriteLine("Quiet electric hum.");
    }
}

class Program
{
    static void Main()
    {
        ElectricCar car = new ElectricCar("Tesla", 80);
        // Output: Vehicle created: Tesla \n ElectricCar ready with 80% battery.
        car.Accelerate(); // Output: Vehicle accelerating. \n Quiet electric hum.
    }
}

```

6. The sealed / final Keywords

C#: sealed class and sealed methods

- sealed class Derived : Base { }: Can't be further inherited (performance/security).
- sealed override Method() { }: Can't be overridden in subclasses.

Preventing Inheritance or Method Overriding: Use for leaf classes (e.g., final utils) or to lock impl.

Example:

```

using System;

public class Vehicle
{
    public virtual void Drive() { Console.WriteLine("Driving"); }
}

public class SportsCar : Vehicle
{
    public override void Drive() { Console.WriteLine("Fast drive"); }
}

public sealed class Ferrari : SportsCar // Sealed class: No further inheritance
{
    public sealed override void Drive() // Sealed method: Can't override

```

```

    {
        base.Drive();
        Console.WriteLine("Vroom!");
    }
}

class Program
{
    static void Main()
    {
        Ferrari f = new();
        f.Drive(); // Output: Driving \n Fast drive \n Vroom!
        // public class Lambo : Ferrari { } // Compile error: Can't inherit sealed
    }
}

```

7. Upcasting and Downcasting

Assigning a Derived Object to a Base Class Reference (Upcasting)

Implicit/safe: Derived ref to base (loses specifics, gains generality).

Casting Back to Derived Type (Downcasting)

Explicit: (Derived)baseRef—throws `InvalidCastException` if wrong.

Safe Casting Using `is` / `as` (C#)

- `is`: Checks type (bool).
- `as`: Casts or null (no exception).
- Pattern matching: `if (obj is Derived d)` (C# 7+).

Example:

```

using System;

public class Animal { public void Eat() { } }
public class Dog : Animal { public void Bark() { } }

class Program
{
    static void Main()
    {
        Dog dog = new();
        Animal animal = dog; // Upcast (implicit)
    }
}

```

```

        // Unsafe downcast
        try
        {
            Dog risky = (Dog)animal;  // OK here
            risky.Bark();
        }
        catch (InvalidCastException)
        {
            Console.WriteLine("Cast failed");
        }

        // Safe: as
        Dog safe = animal as Dog;
        if (safe != null) safe.Bark();  // Output: (Bark called implicitly)

        // Safer: is with pattern
        if (animal is Dog d)
        {
            d.Bark();  // Output: Bark
        }
    }
}

```

8. Constructors and Inheritance

Constructor Chaining (Calling Parent Constructor from Child)

Derived always calls base ctor first (implicit if parameterless).

Implicit and Explicit Constructor Calls

- Implicit: : base() (default ctor).
- Explicit: : base(args).

Parameterized Constructors in Base and Derived Classes

Pass params up; use base for chaining.

Example (Parameterized Chaining):

```

using System;

public class Person  // Base
{
    public string Name;
    public Person(string name)

```



```

    {
        Name = name;
        Console.WriteLine($"Person: {Name} initialized.");
    }
}

public class Employee : Person // Derived
{
    public string Department;
    // Implicit base (error if base has no default)
    public Employee(string name, string dept) : base(name) // Explicit
    {
        Department = dept;
        Console.WriteLine($"Employee in {Department}.");
    }
}

class Program
{
    static void Main()
    {
        Employee emp = new("Alice", "IT");
        // Output: Person: Alice initialized. \n Employee in IT.
        Console.WriteLine(emp.Name); // Alice
    }
}

```

9. Inheritance and Memory Model

How Memory Is Allocated for Base and Derived Class Objects

- **Heap:** Full object size = base + derived (e.g., Animal 8 bytes + Dog 16 bytes = 24 bytes).
- **Stack:** Reference (8 bytes on 64-bit).
- **Shared:** Inherited fields in base portion.

What Happens Internally When an Object of a Derived Class Is Created

1. Heap allocation for composite size.
2. Base fields zero-init.
3. Base ctor runs (sets base fields).
4. Derived fields init.
5. Derived ctor runs.
6. Vtable setup for virtual methods (points to derived impl).

Text Diagram (Memory Layout):

Stack: [Dog ref (8 bytes)] → Heap

Heap (Dog object, 24 bytes):

```
+-----+
| Base (Animal):      | <- 8 bytes (e.g., Name field)
|   Name (string ref)|
+-----+
| Derived (Dog):      | <- 16 bytes (e.g., Breed, virtual table ptr)
|   Breed (string)   |
|   vTable* (overrides)|
+-----+
```

(*vTable*: Virtual method table for polymorphism.)

Example (Verify Size):

```
using System;
using System.Runtime.InteropServices;

public class Animal { public string Name; } // ~16 bytes (ref + padding)
public class Dog : Animal { public string Breed; } // ~24 bytes total

class Program
{
    static void Main()
    {
        Console.WriteLine($"Animal size: {Marshal.SizeOf<Animal>()}"); // Approx 8-16 (unmanaged)
        Console.WriteLine($"Dog size: {Marshal.SizeOf<Dog>()}"); // Larger
    }
}
```

(Note: Exact sizes vary; use for concept.)

10. Abstract Classes vs Inheritance

When to Use Abstract Classes Over Normal Inheritance

- Abstract: When base is incomplete (template/pattern). Forces derived to implement abstracts.
- Normal: Full concrete base.
- Use abstract for shared code + must-customize (e.g., **Stream** abstract for I/O).

Vs Normal: Abstract can't instantiate; enforces contracts.

Implementing Abstract Methods in Derived Classes

abstract in base (no body); override in derived (body required).

Example:

```
using System;
```

```
public abstract class GameCharacter // Abstract base
{
    public string Name { get; set; }
    public abstract void Attack(); // Must implement
    public void Heal() { Console.WriteLine("Healing..."); } // Concrete shared
}

public class Warrior : GameCharacter
{
    public override void Attack()
    {
        Console.WriteLine("Sword swing!");
    }
}

public class Mage : GameCharacter
{
    public override void Attack()
    {
        Console.WriteLine("Fireball!");
    }
}

class Program
{
    static void Main()
    {
        GameCharacter[] chars = { new Warrior { Name = "Aragorn" }, new Mage { Name = "Gandalf" } };
        foreach (var c in chars)
        {
            c.Attack(); // Polymorphic: Sword swing! \n Fireball!
            c.Heal();   // Shared: Healing... (x2)
        }
    }
}
```

11. Interface vs Inheritance

Difference Between Interface Implementation and Class Inheritance

Aspect	Class Inheritance	Interface Implementation
Inherits What	State (fields) + behavior	Behavior contract only (no state)
Keyword	:	, after base (if any)
Instantiation	Base can be concrete	Interfaces can't instantiate
Purpose	"Is-a" hierarchy	"Can-do" capabilities

Multiple Inheritance via Interfaces

`class MyClass : Base, I1, I2 { }`—implements all.

Why Interfaces Solve the “Diamond Problem”

No state/data; pure contracts. Conflicts resolved by explicit impl (e.g., `void I1.Method() { }`).

Example:

```
using System;

interface ILogger { void Log(string msg); }
interface IValidator { bool Validate(string input); }

public class UserService : ILogger, IValidator // Multiple
{
    public void Log(string msg) { Console.WriteLine($"Log: {msg}"); }
    public bool Validate(string input) { return !string.IsNullOrEmpty(input); }
}

class Program
{
    static void Main()
    {
        UserService service = new();
        service.Log("User created"); // Output: Log: User created
        Console.WriteLine(service.Validate("test")); // True
    }
}
```

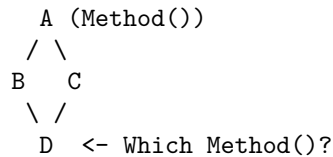
12. The Diamond Problem

What Is It and Why It Occurs in Multiple Inheritance

Ambiguity in multiple paths to same base (e.g., D inherits A via B and C: which A's method?). C# avoids by disallowing class multiple inheritance—prevents duplication/conflict.

Why: State/method ambiguity (e.g., two `A.Method()` versions).

Diagram:



How Interfaces Resolve It

Interfaces: No impl/state by default (C# 8+ defaults OK, but explicit for conflicts). Compiler enforces one impl per signature.

Example (Resolved via Interface):

```
interface IBase { void Method(); }
interface ILeft : IBase { } // Inherit
interface IRight : IBase { }
public class Diamond : ILeft, IRight
{
    void IBase.Method() { Console.WriteLine("Resolved Method"); } // Explicit: One impl
    void ILeft.Method() => ((IBase)this).Method(); // Alias if needed
}
```

13. Practical Examples

Inheritance in Real-World Systems (e.g., Animal → Dog → Puppy)

Multilevel for behaviors.

Example:

```
using System;

public abstract class Animal { public abstract void MakeSound(); }
public class Dog : Animal
{
    public override void MakeSound() { Console.WriteLine("Woof!"); }
}
```

```

public class Puppy : Dog // Multilevel
{
    public override void MakeSound() { Console.WriteLine("Yip! (puppy)"); }
}

class Program
{
    static void Main()
    {
        Animal[] animals = { new Dog(), new Puppy() };
        foreach (var a in animals) a.MakeSound(); // Woof! \n Yip! (puppy)
    }
}

```

Implementing Polymorphic Behavior Using Inheritance

Collection of derived, base calls.

Example (UI Controls):

```

using System.Collections.Generic;

public class Control { public virtual void Render() { } }
public class Button : Control { public override void Render() { Console.WriteLine("Button rendered"); } }
public class TextBox : Control { public override void Render() { Console.WriteLine("TextBox rendered"); } }

class Program
{
    static void Main()
    {
        List<Control> ui = new() { new Button(), new TextBox() };
        ui.ForEach(c => c.Render()); // Polymorphic: Button... \n TextBox...
    }
}

```

Abstract Base Classes for Reusable Design

E.g., HTTP handlers.

Example:

```

public abstract class HttpHandler
{
    public void ProcessRequest() // Template
    {
        Validate(); // Abstract
        Handle();   // Abstract
        Log();       // Concrete
    }
}

```

```

    }
    protected abstract void Validate();
    protected abstract void Handle();
    private void Log() { Console.WriteLine("Request logged"); }
}

public class JsonHandler : HttpHandler
{
    protected override void Validate() { Console.WriteLine("JSON validated"); }
    protected override void Handle() { Console.WriteLine("JSON handled"); }
}

// Usage: new JsonHandler().ProcessRequest(); Output: JSON validated \n JSON handled \n Req

```

14. Best Practices

Favor Composition Over Inheritance (When Applicable)

Use “has-a” (e.g., `Car { Engine engine; }`) for flexibility over “is-a”. Inheritance for true hierarchies; composition for roles (swappable parts).

When: If not strict “is-a” (e.g., Bird “has” FlyBehavior vs. inherits Flyable).

Avoid Deep Inheritance Hierarchies

3 levels max. Use interfaces for cross-cutting. Deep = brittle (base changes ripple).

Keep Base Classes Stable and Generic

- Virtual only when needed (performance hit).
- No derived assumptions (Liskov: Substitutable).
- Stable: Rarely change public APIs.
- Generic: Use constraints (e.g., `where T : Animal`).

Additional C# Tips: - Use records for immutable inheritance (C# 9+). - Attributes for metadata (e.g., `[Obsolete]` on base). - Reflection for runtime inspection (`typeof(Derived).BaseType`). - SOLID: Open-Closed (extend via inheritance/interfaces).

This full C# notes guide is self-contained for mastery. Practice: Build a banking app hierarchy. For expansions (e.g., generics in inheritance), ask!